Full-Stack Engineer — Take-Home Assignment

"Stera Episode Studio"

> **Read this whole document before writing any code.** A meaningful part of this assignment is buried in decisions the spec deliberately does *not* make for you.

---

0. What this is, and how we read it

You're going to build a small full-stack application that ingests raw multimodal recordings, runs them through a real processing pipeline, and presents the results in a web UI.

We are **not** primarily grading whether every feature works. We are grading **your judgment**: how you handle messy real-world data, what you choose to build vs. skip, how you reason about trade-offs, and whether you actually understand the system you submit. A polished app you can't explain scores worse than a rougher app whose every decision you can defend.

On AI tools: Use them. We assume you will, and that's realistic — it's how the job works. But there's a catch you should plan around from minute one:

- The SDK at the center of this task (`stera-sdk`) is a brand-new **alpha** package. AI assistants have little-to-no reliable training data on it and will confidently invent method names, arguments, and return types that do not exist. **You must read the actual docs and source.** Code that calls hallucinated APIs is an instant tell.
- There is a **mandatory live session** (Section 8) where you will walk through your code, make a small change on the spot, and debug a scenario we introduce — with no AI assistance. Anything you submit, you must be able to explain and modify live.

So: AI is allowed and expected. **You own every line.** Build understanding, not just output.

---

1. The product

Stera Episode Studio is an internal tool for a robotics-data team. Researchers record egocentric sessions on an iPhone using the Stera app; each recording is an `.mcap` file containing synchronized RGB, LiDAR depth, 6-DoF camera pose, and IMU streams. Before the data is useful for training, someone has to:

1. **Find** a recording.
2. **Process** it — blur faces (PII), optionally track hands, and export a clean "episode" directory.
3. **Review** the result — watch the video, read the quality report, check whether the recording is even usable.

Right now they do this by hand in a terminal. Your job is to give them a web app.

---

2. The data

You've been given a **Google Drive link** (in your invite email) to a folder of `.mcap` recordings. **Download the files manually** from that link onto your own machine — there is no Drive API and you do **not** need to integrate with Drive. The link is just how we hand you the data; your application works against the local files.

Do not assume the set is clean. It reflects what this team's recordings actually look like. Part of the exercise is discovering what's in there and deciding how your system should react to each thing it finds. How those local files get *into* your app (a configured directory the backend scans, an upload flow in the UI, a CLI ingest step — your call) is itself a design decision; see §5.

The Stera SDK reads these files. Reference docs:

- SDK repo: https://github.com/fpv-labs/stera-sdk
- Docs: https://www.fpvlabs.ai/docs (Process section + API reference)
- PyPI: `pip install "stera-sdk[all]"` — note `ffmpeg` must be on `PATH` for export.

A minimal pipeline (from the SDK quickstart) looks like this:

```python
import stera
from stera import Evaluate
from stera.data import MCAPReader
from stera.models import HandTracker, FaceBlurrer

session = MCAPReader("recording.mcap")          # parses + syncs the streams
blur    = FaceBlurrer(model="mediapipe")
hands   = HandTracker(model="mediapipe")

for frame in session.frames():                  # yields SyncedFrame objects
    clean = blur.blur(frame)
    poses = hands.detect_hands(frame)
    session.add_rgb_frame(frame.index, clean)
    session.add_hand_pose(frame.index, poses)

session.export("episodes/run_01")               # writes the episode directory
Evaluate(session).show()                        # interactive HTML QC report
```

That's the *happy path*. The interesting parts of this assignment are everything the happy path ignores.

---

3. Core requirements (the "must")

Build a web application with a backend and a frontend that lets a user:

1. **Browse recordings.** List the recordings available to the app (from the local files you downloaded) with whatever metadata is cheap to surface before processing (name, size, etc.). How recordings get into the app is your design call (see §5).
2. **Trigger processing** on a selected recording. At minimum your pipeline must:
   - open the recording with `MCAPReader`,
   - **blur faces** on every RGB frame (this is non-negotiable — see §5),
   - export an episode directory via `session.export(...)`,
   - generate the quality report via `Evaluate(session)`. Hand tracking and skeleton/mesh steps are optional — your call (and justify it).
3. **Watch jobs run.** Processing a recording is slow. The UI must show progress and not freeze or block. How you do this (polling, websockets, SSE, a job queue) is up to you.
4. **Review results.** For a processed recording show, at minimum: the exported video, the QC report / health score, and key metadata (duration, frame counts, camera intrinsics, which streams were present). 3D map / point-cloud viewing is a bonus, not required.
5. **Persist** processed results so reopening a recording doesn't reprocess it.

Everything else — auth, styling, framework choices, deployment shape — is your call. We care more about the spine working end-to-end than about breadth.

---

4. Stretch (only if the core is solid — do NOT gold-plate)

Pick **at most one or two** of these, or propose your own. We would rather see a clean core than a sprawling, half-working pile of features.

- Live 3D viewer for the exported map / point cloud / mesh.
- Multi-recording dashboard: health scores across all sessions, sortable.
- Re-run with different parameters (e.g. tighter sync windows) and compare reports.
- Cancellation / resumability of long jobs.

> If you spend time on stretch goals while the core is shaky, that counts **against** you. Prioritization is part of the test.

---

5. The judgment tasks (read carefully — this is where the signal is)

The spec above is deliberately under-specified. The following situations *will* come up. We are not telling you what the "right" behavior is, because there often isn't one — there's a

defensible decision and an indefensible one. **Detect each situation, decide how to handle it, and record the decision and your reasoning in `DECISIONS.md` (§6).**

1. **Not every recording is well-formed.** Some are missing streams (e.g. no depth, no map mesh), some have streams that don't line up cleanly in time, and the folder may contain things that aren't valid recordings at all. What should your pipeline and your UI do in each case? When do you fail loudly vs. degrade gracefully? (The SDK's `MCAPReader` has a `check_format` flag and `frames()` has `max_depth_dt` / `max_pose_dt` sync windows — understand what they actually do before you set them.)
2. **At least one recording is large.** Stera sessions can run well over an hour at full frame rate. A pipeline that loads every frame into memory before processing will fall over. Show us you thought about memory and time, not just correctness on a 10-second clip.
3. **Faces are PII.** Real human faces appear in this footage. No un-blurred frame should ever reach the browser, be written to a public path, or sit in a cache an unauthenticated request could reach. Treat this as a correctness requirement, not a nice-to-have. (We will specifically check this.)
4. **The SDK will surprise you.** It's alpha. Some calls are slow, some are heavier than they look, return types may not match your first guess, and an optional model backend may not install cleanly. When something doesn't behave as documented, we want to see how you investigate and what you decide — not a silent `try/except: pass`.
5. **Getting files into the app is not free.** These recordings are real binary files and at least one is large (see #2). Whether you ingest via a configured directory, an upload flow, or a CLI step, the naive version has sharp edges: validating that an uploaded/scanned file is actually a usable recording before you queue work on it, not choking the request/UI on a multi-hundred-MB file, and not silently accepting a half-copied or junk file. We want to see you noticed them.

In your `DECISIONS.md`, we expect to see **specific, data-grounded** answers — things that are only true if you actually ran the pipeline on *these* files. Generic answers read as "I never opened the data."

---

6. Deliverables

Submit a **private Git repository** (invite the address in your email) containing:

1. **The application** — backend + frontend, in one repo.
2. `README.md` with setup and run instructions that **work from a cold clone**. We will follow them exactly on a clean machine. "Works on my machine" is a fail mode here. Containerize if that's the most reliable path.
3. `DECISIONS.md` — your design journal. This is weighted as heavily as the code. We want:
   - The architecture in a few sentences + a sketch (ASCII/diagram fine).
   - Every non-obvious decision, the alternative you rejected, and *why*.
   - Your answers to the §5 judgment tasks, grounded in the actual data.

    - **Where and how you used AI tools, and how you verified the output.** Being specific here is a plus, not a confession.
    - "What I'd do with another day" and "What I knowingly cut."
4. `KNOWN_ISSUES.md` (can be a section of the README) — an honest list of what's broken, flaky, or unfinished. We trust a candidate who tells us where the bodies are buried far more than one who pretends there are none. **Hiding a known bug is worse than having it.**
5. **A short screen-recording (≤ 5 min)** walking through the app running on the *actual provided data* — not a toy file. A link (Loom/YouTube-unlisted/file) is fine.
6. **Real commit history.** Commit as you go, in meaningful increments, with messages that explain intent. A single squashed "final" commit removes a signal we rely on and counts against you.

---

7. Constraints & freedom

- **Language/stack:** The pipeline is Python (the SDK is Python-only), so your backend will almost certainly be Python. Frontend and everything else: your choice. Use what you're fastest and most confident in — you'll be defending it live.
- **Scope of "full-stack":** We're looking at backend design, async/long-job handling, data handling, a usable frontend, and the integration glue that makes it run as one system. You don't need production auth, multi-tenancy, or fancy design.
- **Third-party libraries / scaffolding:** Fine. Just understand what they do.

---

8. The live session (mandatory — plan for it)

After you submit, we'll book a **45–60 minute** call. We'll run your app from your README first, then:

- You walk us through the architecture and a couple of functions **you** wrote.
- We ask you to make a **small change or extension live** (screen-share, your editor, **no AI assistant**).
- We introduce a **scenario or bug** and watch how you reason through it.
- We ask "why X and not Y" about a few of your decisions.

The point isn't to trip you up — it's that someone who genuinely built and understood this will sail through, and that's the strongest signal we have. **Build for understanding accordingly.**

---

9. Time and expectations

- Target **6–8 focused hours**. This is a sizing signal, not a race; we are not measuring raw speed and we don't want your weekend. If you hit the wall, **stop and document what you'd do next** in `DECISIONS.md` — that's worth more than a rushed, unexplained feature.
- A smaller, coherent, fully-explained submission beats a large, shaky one. Every time.

---

10. How you'll be evaluated (high level)

Area	What we're looking for
Judgment & decisions	The §5 situations handled thoughtfully; `DECISIONS.md` shows real reasoning and real engagement with the actual data.
End-to-end correctness	The core pipeline genuinely runs on the provided recordings and produces a reviewable result.
System design	Sensible separation of concerns; long jobs handled without blocking; results persisted.
Data & ops sense	Memory/time awareness on big files; graceful handling of malformed input; PII respected; cold-clone reproducibility.
Communication	README that works, honest KNOWN_ISSUES, clear commits, and a live session where you own your code.

Good luck. Read the data before you trust the spec.